

Prediccion del Tier de Sala de Artistas de Rap Urbano Espanol

Victor | Master en Data Science
Junio 2026

182

artistas
del rap/urbano
espanol

68.2%

accuracy
(CV5 estratificado)

6 fuentes

de datos
Spotify, YT, Last.fm
setlist.fm, GTrends

Contenido del guion

1. Contexto y motivacion del proyecto
2. Diseno del sistema de tiers (variable objetivo)
3. Pipeline de datos: 6 fuentes, 32 features
4. Exploracion y seleccion de variables (EDA)
5. Comparativa de modelos y seleccion
6. XGBoost: como funciona y por que decide lo que decide
7. Ajuste de hiperparametros y resultado final
8. Explicabilidad con SHAP
9. Dashboard Streamlit y agente IA
10. Valor de negocio y casos de uso
11. Preguntas del tribunal: anticipadas y respuestas
12. Conclusiones y lineas futuras

1. Contexto y Motivacion del Proyecto

El sector de la musica en directo en Espana mueve mas de 1.000 millones de euros anuales (SGAE, 2023). Decidir en que sala actua un artista emergente es hoy una decision que se toma principalmente por intuicion: el equipo de management evalua la trayectoria, habla con promotores y compara con referencias del sector. El error tiene coste directo: una sala sobredimensionada genera perdidas y dania la imagen del artista; una infradimensionada deja ingresos y fans en la puerta.

Pregunta de investigacion: ¿Es posible predecir de forma objetiva, a partir de datos publicos de plataformas digitales e historial de conciertos, el tamano de sala que un artista de rap/urbano espanol puede llenar?

Por que rap/urbano espanol?

Escena especialmente adecuada para este analisis

- Genero en rapido crecimiento: Quevedo, Bad Gyal, Morad son fenomenos de audiencia masiva reciente.
- Ecosistema muy heterogeneo: desde artistas underground con 500 oyentes hasta nombres con 10M+ en Spotify.
- Datos publicos disponibles: setlist.fm tiene cobertura razonable de la escena; Last.fm registra scrobbles historicos.
- 182 artistas identificados y etiquetados manualmente por el autor, con conocimiento directo de la escena.

Por que este TFM aporta valor?

- Es el primer modelo de ML especifico para artistas emergentes del rap/urbano espanol.
- Integra 6 fuentes de datos heterogeneas en un pipeline reproducible y extensible.
- Incluye un dashboard interactivo con explicabilidad SHAP y un agente IA de seguimiento.
- El etiquetado semisupervisado (setlist.fm + conocimiento experto) es una contribucion metodologica propia.

2. Diseno del Sistema de Tiers (Variable Objetivo)

La variable objetivo es el **tier de sala**: el tamaño máximo de recinto que un artista puede llenar en España. Se definió en 3 niveles con criterio de aforo real:

Tier	Aforo	Salas típicas	Ejemplos	N
BAJO (1)	< 200	Sala Vibora, locales underground	Tarchi, Gatti, Xico Palma	81
MEDIO (2)	200 - 2.000	El Sol, Razzmatazz, La Riviera, Copera	BEJO, La Zowi, Choclock, Metrika	62
ALTO (3)	> 2.000	WiZink, Movistar Arena, festivales	Bad Gyal, Quevedo, Morad, Rels B	39

Proceso de etiquetado semisupervisado

Con 182 artistas no existe un dataset público etiquetado. Se diseñó un proceso híbrido:

- ~50% de las etiquetas se infirieron automáticamente desde el historial de venues de setlist.fm (script `generar_labels.py`): si el venue máximo registrado supera cierto aforo, se asigna el tier correspondiente.
- El 50% restante se etiquetó manualmente con conocimiento directo de la escena musical española.
- Resultado: 182 artistas etiquetados, distribución desbalanceada natural (bajo=81, medio=62, alto=39).

Por qué 3 clases y no más?

Los tiers originales (0-4) se fusionaron en 3 niveles por una razón estadística fundamental: con $n=182$ artistas, tener 5 clases implicaría ~36 muestras por clase de media, insuficiente para que los modelos aprendan patrones robustos. 3 clases garantiza >39 ejemplos en la clase minoritaria (alto), manteniendo la interpretabilidad ordinal.

3. Pipeline de Datos: 6 Fuentes, 32 Features

El pipeline recoge senales de presencia digital, audiencia historica y actividad en directo. Cada fuente aporta una dimension diferente de la trayectoria del artista:

Fuente	N. Features	Que mide	Reto tecnico
Spotify (discografia)	6	Madurez de carrera: anos activo, volumen y cadencia de lanzamientos	popularity y audio_features bloqueados desde nov 2024 (403)
Spotify (top tracks)	3	Estilo: duracion, % explicit, % colaboraciones	Requiere top-10 por artista; normalizacion de duraciones
Last.fm	6	Audiencia historica: oyentes unicos, scrobbles, engagement fan	Fuzzy matching de nombres (umbral 0.70) para artistas con alias
YouTube	7	Presencia digital: suscriptores, vistas, engagement del canal	41% de canales requirieron correccion manual en el registry
setlist.fm	6	Actividad en directo: conciertos, paises, % en Espana	41% de NaN — artefacto detectado y corregido con flag <code>sl_tiene_datos</code>
Google Trends + YT reciente	5	Buzz actual: interes de busqueda y vistas recientes	Rate-limiting de pytrends; TikTok sin API publica accesible

Registry de artistas: la pieza clave

El principal reto no es el modelado sino la **ingenieria de datos**: cada plataforma tiene nombres e IDs distintos para el mismo artista. Se construyo un *artistas_registry.csv* como fuente de verdad unica que mapea el nombre canonico a los IDs especificos de cada plataforma. Los colectores nunca buscan por nombre — usan el ID directamente, eliminando falsos positivos de matching.

- `manual_override=True` protege las correcciones manuales de sobreescrituras automaticas.
- Pipeline incremental: si un artista ya tiene datos, se salta. Los datos validos nunca se pierden.
- 114 tests unitarios verifican todo el pipeline desde la ingesta hasta la prediccion.

Artefacto critico detectado: setlist.fm y el 41% de NaN

Problema detectado y resolucion aplicada

- El 41% de artistas no aparece en setlist.fm. El script validar_setlistfm.py confirmo que este NaN NO es aleatorio.
- Chi-cuadrado: $\chi^2=63.3$, $p \sim 0.000$ -- el 79% de artistas 'bajo' no tiene datos en setlist.fm vs solo 5.3% de 'alto'.
- Si imputamos $\text{NaN} \rightarrow 0$, el modelo aprende implicitamente que $\text{sl_num_conciertos}=0$ es senal de nivel bajo -- un artefacto.
- Solucion: se anadio `sl_tiene_datos` (flag binario 0/1 creado ANTES de imputar), haciendo explicita la distincion 'sin perfil' vs '0 conciertos'.
- Validacion: eliminar todas las features de setlist.fm cuesta -6.4 puntos de F1 macro -- la senal es real, solo habia que tratarla honestamente.

4. Exploracion y Seleccion de Variables (EDA)

El EDA (notebook 01_eda.ipynb) analizo las 36 features candidatas usando **Kruskal-Wallis** (discriminacion entre clases, no asume normalidad) y **correlacion de Spearman** (relacion monotona con el target):

Feature	H (KW)	r (Spearman)	Interpretacion
lfm oyentes	61.6	+0.63	Feature mas discriminativa: audiencia historica acumulada
lfm_scrobbles	59.2	+0.62	Engagement acumulado -- correlacion alta con oyentes
yt_suscriptores	53.3	+0.59	Presencia digital: canal de YouTube como proxy de popularidad
yt_vistas_totales	51.5	+0.57	Muy alta senal: consumo acumulado en YouTube
sl_num_conciertos	--	+0.47	Feature #1 en importancia de modelos (arbol)
sp_num_eps	~0	~0	Sin varianza util -- eliminada
yt_edad_canal_anos	~1	+0.10	No significativa -- eliminada

Resultado: 27 de 36 features son significativas (Kruskal-Wallis $p < 0.05$). Se eliminaron 5 features por baja senal:

- sp_num_eps -- sin varianza: casi todos los artistas tienen 0 EPs.
- sp_num_total_releases -- redundante: suma de albums+singles+eps.
- sp_dias_desde_ultimo_release -- $H \sim 2$, $r = +0.10$ -- no discrimina niveles.
- yt_edad_canal_anos -- $H \sim 1$ -- la antigüedad del canal no predice el tier.
- trend_yt_videos_recientes -- varianza casi nula: >95% tienen 5 videos recientes.

Tratamiento de outliers extremos

Quevedo, Bad Gyal y Morad tienen valores de YouTube 10-50x superiores a la mediana del tier ALTO. Se mantienen como datos validos (son la realidad del mercado, no errores). Para modelos lineales se aplica **RobustScaler** (usa mediana e IQR en lugar de media y desv. típica), que mitiga su efecto sin eliminarlos.

5. Comparativa de Modelos y Selecccion

Se evaluaron 8 modelos con **StratifiedKFold k=5**. Con solo 182 artistas, un hold-out del 20% dejaria ~36 muestras de test, estadisticamente insuficiente. Con CV k=5 cada artista aparece exactamente una vez en test, aprovechando todos los datos sin data leakage.

Modelo	Features	Accuracy CV5	F1 macro CV5	Estabilidad
Dummy most_frequent	--	44.5% +/- 0.9%	20.5% +/- 0.3%	Baseline
Regresion Logistica	24 (lineal)	59.4% +/- 7.5%	56.3% +/- 8.1%	Variable
Regresion Ordinal (mord)	24 (lineal)	60.4% +/- 4.9%	56.4% +/- 4.5%	Estable
Random Forest	32 (arbol)	61.6% +/- 3.2%	58.5% +/- 3.5%	Muy estable
LightGBM	32 (arbol)	61.0% +/- 5.7%	57.6% +/- 5.0%	Variable
XGBoost base	32 (arbol)	63.2% +/- 3.9%	60.3% +/- 3.7%	Estable
SVM (kernel RBF)	24 (lineal)	64.8% +/- 7.6%	62.3% +/- 8.0%	Variable
XGBoost tuned	32 (arbol)	68.2% +/- 5.0%	66.6% +/- 6.3%	Estable

Por que XGBoost tuned como modelo final?

- Mayor F1 macro (66.6%) -- supera al SVM base (62.3%) tras el tuning.
- Estabilidad: desv. tipica de 5.0% en accuracy frente al 7.6% del SVM.
- Explicabilidad nativa con SHAP TreeExplainer -- fundamental para el dashboard.
- Invariante al escalado -- no necesita RobustScaler, simplificando el pipeline de inferencia.
- El F1 macro sube +46 puntos sobre el baseline dummy (20.5%) -- la mejora es sustancial.

Analisis de errores del mejor modelo

64 de 182 artistas mal clasificados en OOF. La clase MEDIO es la mas dificil ($F1=0.55$): los errores son simetricos (~12 clasificados como BAJO, ~15 como ALTO). Solo 7 errores de 2 saltos (ALTO->BAJO o BAJO->ALTO): el modelo respeta la ordinalidad implicitamente aunque no se le imponga de forma explicita.

6. XGBoost: Como Funciona y Por que Decide lo que Decide

Intuicion del algoritmo (sin formulas)

XGBoost es un ensamble de **arboles de decision poco profundos** que se construyen de forma secuencial. Cada arbol nuevo intenta corregir los errores que cometio el anterior. Con los hiperparametros finales, el modelo usa **400 arboles de profundidad maxima 2** -- lo que significa que cada arbol solo puede hacer **2 preguntas sobre los datos** (por ejemplo: '¿tiene mas de 500 oyentes en Last.fm?' y '¿ha actuado en mas de 3 paises?'). La prediccion final es el promedio ponderado de todos los arboles.

Por que max_depth=2?

Con 182 artistas, arboles profundos aprenden de memoria los datos de entrenamiento (overfitting). Al limitar la profundidad a 2, el modelo esta obligado a encontrar las 2 variables que MAS separan los niveles en cada etapa, en lugar de memorizar combinaciones especificas de artistas concretos. Los 400 arboles de profundidad 2 combinados capturan interacciones complejas sin sobreajustarse.

Logica de decision: que mira el modelo

Las features importances (RF y XGBoost en consenso) revelan la jerarquia de decision:

- **sl_num_conciertos** (feature #1): Un artista con muchos conciertos documentados casi siempre es MEDIO o ALTO. Sin historial de directos, el modelo tiende a predecir BAJO (aunque con sl_tiene_datos ahora distingue 'sin perfil' de '0 conciertos').
- **lfm_oyentes / lfm_oyentes_log**: La audiencia historica en Last.fm es la senal mas limpia ($r=+0.63$ con el target). Artistas con >100.000 oyentes historicos raramente son BAJO.
- **yt_vistas_totales / yt_suscriptores**: Presencia digital acumulada. Un canal sin vistas es senal fuerte de BAJO; millones de vistas apuntan a ALTO.
- **sl_num_paises**: Actuar en varios paises es un salto cualitativo -- casi todos los ALTO han actuado fuera de Espana.
- **lfm_scrobbles**: El engagement acumulado complementa el volumen de oyentes -- pocos oyentes con muchos scrobbles indica base de fans muy fiel.

Como se genera la probabilidad de clase

El modelo usa **multi:softprob** como objetivo: para cada artista, calcula 3 scores (uno por clase) y los pasa por una funcion softmax, obteniendo probabilidades que suman 1.0. La clase predicha es el argmax (la clase con mayor probabilidad). El dashboard muestra las 3 probabilidades, lo que permite detectar casos de alta incertidumbre: si BAJO=0.48, MEDIO=0.35, ALTO=0.17, la prediccion es BAJO pero el modelo no esta seguro.

Regularizacion: por que el tuning ayudo tanto

Hiperparametros clave y su efecto

- `learning_rate`: 0.05 -> 0.01 -- aprende mas despacio, generaliza mejor (necesita mas arboles).
- `min_child_weight`: 1 -> 10 -- exige al menos 10 muestras en cada hoja final, evitando splits sobre 1-2 artistas.
- `gamma`: 0 -> 0.5 -- requiere una ganancia minima de informacion para hacer un split.
- `reg_alpha` (L1): 0 -> 1.0 y `reg_lambda` (L2): 1 -> 2.0 -- penalizan pesos grandes, reducen overfitting.
- `max_depth`: 4 -> 2 -- arboles mas simples, fuerzan al modelo a usar solo las features mas discriminativas.
- `colsample_bytree`: 0.8 -> 0.5 -- cada arbol ve solo la mitad de las features, aumentando diversidad del ensamble.
- Patron global: TODOS los cambios apuntan a MAS regularizacion -- la busqueda automatica detecto el riesgo de overfitting con $n=182$.

7. Ajuste de Hiperparametros y Resultado Final

Se uso **RandomizedSearchCV** con 100 combinaciones aleatorias x CV5 estratificado = 500 fits. Metrica de optimizacion: **F1 macro** (penaliza errores en clases minoritarias, mas adecuado que accuracy con clases desbalanceadas).

Modelo	Accuracy CV5	F1 macro CV5	Estabilidad
XGBoost base	63.2% +/- 3.9%	60.3% +/- 3.7%	Estable
XGBoost tuned	68.2% +/- 5.0%	66.6% +/- 6.3%	Estable
Delta	+5.0 puntos	+6.3 puntos	--

El delta de +6.3 puntos en F1 macro **supera el umbral de interpretabilidad** (~3 puntos con n=182): la mejora es real y no ruido estadistico. El modelo tuneado supera tambien al SVM base que habia liderado el benchmark sin tuning (62.3% F1).

El modelo serializado (*models/xgb_tuned.joblib*) y su contrato de features (*models/metadata.json*) estan en git, garantizando reproducibilidad completa.

8. Explicabilidad con SHAP

SHAP (SHapley Additive exPlanations) responde a '¿por que el modelo predijo este tier para este artista?' de forma matematicamente rigurosa. A diferencia de la importancia de features (que es global y no causal), SHAP descompone cada prediccion en la contribucion de cada feature individual.

Como interpretar un valor SHAP

Cada valor SHAP indica cuanto empuja esa feature la prediccion hacia arriba o hacia abajo respecto al **valor base** del modelo (la probabilidad media de la clase en el dataset). Por ejemplo, si el valor base para ALTO es 0.21 y el artista tiene sl_num_conciertos muy alto, ese feature puede empujar la probabilidad hasta 0.65.

- **SHAP positivo:** ese valor de feature empuja la prediccion HACIA la clase analizada.
- **SHAP negativo:** ese valor de feature aleja la prediccion DE la clase analizada.
- **SHAP = 0:** esa feature no tiene efecto en esta prediccion concreta.

Plots globales (todo el dataset)

- **Bar plot (shap_summary_bar.png):** media del |SHAP| por feature, agregada sobre todas las clases. Mas robusto que la importancia MDI de Random Forest ante features correlacionadas.
- **Beeswarm para clase ALTO (shap_beeswarm_alto.png):** cada punto es un artista. El eje X muestra cuanto empuja esa feature la prediccion hacia ALTO. Los artistas con alto sl_num_conciertos (puntos rojos a la derecha) se predicen como ALTO.

Waterfall individual (por prediccion)

El dashboard genera un waterfall plot en tiempo real para cada artista analizado. Muestra las 15 features que mas contribuyeron al resultado, con flechas de izquierda (alejan de la clase) o derecha (acercan a la clase). Esto permite al usuario entender exactamente por que el modelo predijo ese tier y que variables deberia mejorar para subir al siguiente nivel.

Valor de negocio de SHAP

SHAP convierte el modelo en una herramienta accionable

- Un manager puede ver: 'sl_num_conciertos bajo es la razon principal por la que el modelo predice BAJO'.
- Permite priorizar acciones: 'aumenta presencia en directos y Last.fm antes que lanzar mas singles'.
- Genera confianza en el modelo: el tribunal o el cliente puede ver que las razones tienen sentido musical.
- Detecta casos anormales: si SHAP muestra que yt_vistas_totales pesa mucho pero las vistas son bajas, puede haber un error de datos.

9. Dashboard Streamlit y Agente IA

El dashboard convierte el modelo en una herramienta usable por perfiles no tecnicos. Disponible en: datascience-masterthesis-rszvqcfaehgxdhnxbaaz3a.streamlit.app

Pagina	Contenido	Publico objetivo
Landing	KPIs del modelo, sistema de tiers, navegacion	Cualquier perfil
Resultados	Benchmark de 8 modelos, confusion OOF, feature importance RF/XGBoost/SHAP global	Tecnico / tribunal
Prediccion	Flujo automatico: nombre -> APIs -> datos editables -> prediccion + SHAP waterfall + alertas	Manager / promotor / artista
Analisis IA	Agente LangChain+Groq: explicacion estructurada + chat de seguimiento	Negocio / gestion

Flujo automatico de prediccion

- El usuario escribe el nombre del artista -- Spotify busca y auto-confirma si el match es claro (score $\geq 90\%$).
- Si hay empate, el LLM (Llama 3.3 70B via Groq) sugiere cual es el artista de rap espanol.
- 5 APIs se consultan en paralelo (ThreadPoolExecutor) -- el progreso se muestra en tiempo real.
- Todos los valores son editables antes de predecir -- el usuario puede corregir cualquier dato.
- Tras la prediccion: probabilidades de las 3 clases, waterfall SHAP y alertas de inconsistencias.

Agente IA (LangChain + Groq)

El agente recibe como contexto: las features del artista, el tier predicho con probabilidades, los valores tipicos por tier del dataset y artistas de referencia. Genera automaticamente una explicacion en 3 secciones: '¿por que este tier?', 'confianza de la prediccion' y 'que deberia mejorar'. El chat de seguimiento permite preguntas libres sobre la prediccion.

10. Valor de Negocio y Casos de Uso

El modelo responde a una necesidad real del sector: reducir el riesgo en las decisiones de contratacion y programacion de giras. El coste de error es directo y cuantificable.

Usuarios objetivo

Perfil	Caso de uso	Valor generado
Management artistico	Evaluar si el artista esta listo para dar el salto de sala pequeña a media	Evitar apostar por sala incorrecta; optimizar momento de la gira
Promotores booking /	Screening inicial de artistas emergentes sin historial de contratacion	Reduce tiempo de evaluacion; base objetiva para negociacion
Festivales	Identificar artistas en tier BAJO/MEDIO con perfil de crecimiento hacia ALTO	Detectar talentos antes de que exploten, a mejor precio
Sellos discograficos	Evaluar potencial de artistas en proceso de firma	Dato adicional objetivo en decisiones de A&R;

Limitaciones de negocio que el modelo no captura

- Calidad artistica: el modelo mide presencia digital, no si la musica es buena.
- Eventos externos: un featuring viral o una sincronizacion en serie puede disparar el tier overnight.
- Artistas que evitan deliberadamente las plataformas digitales (underground hardcore).
- Nuevos artistas sin datos historicos: el modelo no puede predecir desde cero.
- El tier predice el maximo posible, no el momento optimo para dar el salto.

Como se usaria en produccion

El pipeline es incremental y reproducible. Para integrar un nuevo artista: 1) añadir a artistas.txt, 2) resolver identidades con el script automatico, 3) ejecutar pipeline.py (recolecta, construye features, re-entrena si hay suficientes artistas nuevos). El dashboard siempre refleja el modelo mas reciente via el .joblib en git.

11. Preguntas del Tribunal: Anticipadas y Respuestas

T: ¿Por que no usas un conjunto de test separado para evaluar el modelo final?

Con 182 artistas, un hold-out del 20% dejaria solo 36 muestras de test -- estadisticamente insuficiente para estimaciones fiables con 3 clases. StratifiedKFold k=5 garantiza que cada artista aparece exactamente una vez en test y preserva la proporcion de clases en cada fold. Es la estrategia estandar de la literatura para datasets pequeños. Las metricas reportadas (68.2% acc, 66.6% F1) son OOF -- fuera de muestra -- no de entrenamiento.

T: ¿68% de accuracy no es bajo para un modelo de ML?

Depende del baseline. El dummy most_frequent (siempre predice 'bajo', la clase mayoritaria) alcanza 44.5%. Nuestro modelo llega a 68.2%, es decir, +23.7 puntos sobre el azar informado. En F1 macro la mejora es aun mas dramatica: +46 puntos sobre el baseline (20.5% -> 66.6%). Ademias, con solo 182 artistas, 3 clases y datos de plataformas con mucho ruido, este rendimiento es competitivo. Lo importante es que el modelo respeta la ordinalidad (solo 7 errores de 2 saltos sobre 182 artistas).

T: ¿No es circular usar setlist.fm para etiquetar Y y tambien como feature en X?

Es una pregunta clave. La variable objetivo se etiqueta desde el historial de venues (el aforo del mayor venue donde ha actuado), mientras que las features de setlist.fm capturan el VOLUMEN de actividad (numero de conciertos, paises, etc.). Son dimensiones diferentes: puedes haber actuado mucho en salas pequeñas (BAJO con sl_num_conciertos alto) o poco en salas grandes (ALTO con sl_num_conciertos bajo). El 41% de NaN confirma que muchos artistas bajo no tienen datos -- el flag sl_tiene_datos hace explicita esta distincion para que el modelo no aprenda un artefacto.

T: ¿Por que XGBoost y no SVM, que tenia mayor F1 sin tuning?

Tres razones: primero, tras el tuning XGBoost supera al SVM base (66.6% vs 62.3% F1). Segundo, el SVM tiene desv. tipica de 7.6% en accuracy -- alta varianza que no da confianza. Tercero, y mas importante: XGBoost tiene explicabilidad nativa con SHAP TreeExplainer, mientras que el SVM RBF es una caja negra para la que SHAP seria mucho mas costoso computacionalmente. Para un proyecto orientado a negocio, la explicabilidad es un requisito no negociable.

T: El dataset tiene 182 artistas. ¿No es demasiado pequeno para ML?

Es un dataset pequeno, pero no el contexto incorrecto para ML. Los artistas de rap/urbano espanol con trayectoria suficiente para ser etiquetados son aproximadamente estos 182 -- no es que falten datos, es que la poblacion es pequena. La respuesta correcta no es inventar datos sino adaptar la metodologia: CV5 en lugar de hold-out, regularizacion agresiva en el modelo, y metricas OOF en lugar de train. El resultado es un modelo que generaliza razonablemente bien a nuevos artistas dentro del mismo genero y mercado.

T: ¿Como afecta que Spotify bloquee popularity y audio_features en noviembre 2024?

Es una limitacion real y significativa. popularity habria sido probablemente la feature mas discriminativa por ser la metrica mas directa de popularidad en Spotify. Se compenso con: (1) oyentes y scrobbles de Last.fm como proxy de audiencia historica, (2) suscriptores y vistas de YouTube como señal de popularidad digital, (3) Google Trends para el buzz actual. El modelo resultante probablemente infravalora artistas muy populares en Spotify pero con poca presencia en Last.fm o YouTube -- un sesgo a documentar como limitacion.

T: ¿Por que no incluir TikTok? Es la plataforma clave para artistas emergentes.

Completamente de acuerdo en que TikTok es hoy la principal plataforma de viralizacion para este genero. No se incluyo por una razon tecnica concreta: la TikTok Research API requiere aprobacion institucional (solo para universidades acreditadas), y las librerias no oficiales (TikTokApi) van contra los Terminos de Servicio -- usarlas en un TFM academico no es aceptable metodologicamente. YouTube reciente se uso como proxy. Es la principal linea futura de mejora del proyecto.

T: ¿Como se valida que el etiquetado manual es correcto?

El etiquetado manual partio del conocimiento directo del autor de la escena, pero se validó de tres formas: (1) el ~50% automatico desde setlist.fm fue consistente con el 50% manual en los artistas con solape, (2) artistas de referencia claros (Quevedo=ALTO, Tarchi=BAJO) actuan como anclas para calibrar el criterio, (3) el modelo aprende patrones coherentes con la intuicion del sector (sl_num_conciertos y lfm_oyentes como features top), lo que sugiere que el etiquetado captura señal real y no ruido.

12. Conclusiones y Lineas Futuras

Conclusiones principales

- Es posible predecir el tier de sala de artistas de rap/urbano español con 68.2% de accuracy y 66.6% de F1 macro usando solo datos publicos de APIs, +46 puntos sobre el baseline dummy.
- La ingeniería de datos (registry, etiquetado semisupervisado, detección del artefacto de setlist.fm) fue el reto mas complejo del proyecto -- mas que el modelado en si.
- XGBoost con regularización agresiva es el modelo mas adecuado para datasets pequeños ordenables, especialmente cuando la explicabilidad es un requisito.
- SHAP convierte el modelo en una herramienta accionable: el usuario no solo recibe un tier sino las razones concretas y las variables que debería mejorar.
- El agente IA (LangChain + Groq) democratiza el acceso al modelo para perfiles no técnicos, trasladando la predicción al lenguaje del sector musical.

Limitaciones principales

- n=182 artistas: tamaño pequeño que limita la complejidad del modelo y la estabilidad de las métricas.
- Sin TikTok: la plataforma mas relevante para artistas emergentes no tiene API accesible.
- Spotify popularity bloqueado: la métrica mas directa de popularidad en streaming no esta disponible.
- Sesgo temporal: los datos reflejan el estado de las plataformas en mayo 2026 -- el modelo envejece.
- Genero específico: los patrones aprendidos pueden no transferirse a otros generos musicales.

Lineas futuras de mejora

Linea	Descripcion	Impacto esperado
TikTok API	Integrar vistas y viralidad en TikTok cuando la API sea accesible	Alto -- principal señal de viralidad actual
Ampliar dataset	Incluir otros generos (pop, electronica, indie) con etiquetado especifico	Alto -- permite generalizar el modelo
Reentrenamiento periodico	Pipeline automatico mensual para actualizar features y reentrenar	Medio -- mantiene el modelo actualizado
Modelo ordinal explicito	Probar mord.LogisticAT o XGBoost con loss ordinal para explotar mejor la estructura	Medio -- podria reducir errores de 2 saltos
Embeddings de audio	Si Spotify reactiva audio_features o se usa otra fuente (AudD, ACRCLOUD)	Alto -- señal de calidad artistica
Series temporales	Modelar la evolucion del tier en el tiempo, no solo el estado actual	Alto -- detectaria artistas en ascenso

Linea	Descripcion	Impacto esperado
Interfaz multi-artista	Comparativa lado a lado de varios artistas en el dashboard	Bajo-Medio -- valor para booking y festivales

Reflexion final

Este proyecto demuestra que las tecnicas de Machine Learning pueden aportar valor real en industrias creativas donde las decisiones se toman historicamente por intuicion. No como sustituto del juicio experto -- un modelo no captura la calidad artistica, el carisma en el escenario o el momentum cultural -- sino como una herramienta de apoyo que aporta objetividad, reproducibilidad y transparencia a decisiones que hoy se toman con informacion incompleta. La explicabilidad con SHAP y el agente IA son los elementos que hacen que el modelo sea util mas alla del laboratorio.